

Tipos de dato y Sistema de tipos

Introducción

Un **tipo de dato** define una colección de valores y un conjunto de operaciones permitidas sobre los valores del tipo.

Además de los tipos predefinidos (enteros, punto flotante, caracter, boolean, string), los lenguajes brindan constructores que permiten a los usuarios definir sus propios tipos, acorde a sus necesidades (enumerados, arreglos, arreglos asociativos, registros, entre otros).

Cada tipo tendrá sus propias decisiones de diseño asociadas.

El **sistema de tipos** es un conjunto de reglas que estructuran y administran una colección de tipos, incluyendo reglas de compatibilidad y conversión, equivalencia, inferencia, etc.

Tipos de datos

Veamos el manejo de tipos, incluyendo de qué manera resuelven los chequeos, que hacen Python y TypeScript.



Python

- Admite programación imperativa, orientada a objetos y funcional.
- Altamente flexible.
- Lenguaje interpretado.
- **Tipado dinámico.**
- **Fuertemente tipado.**



TypeScript

- Admite programación imperativa, orientada a objetos y funcional.
- Mejora limitaciones de JavaScript.
- Lenguaje compilado.
- **Tipado estático.**
- **No fuertemente tipado.**

Caso de estudio: Python

Introducción



Python es un lenguaje de scripting de propósito general, altamente flexible.

Admite programación siguiendo los paradigmas imperativo, orientado a objetos y funcional.

Es un lenguaje interpretado con tipado dinámico, **fuertemente tipado**.

Los chequeos de tipos se realizan en tiempo de ejecución.

La estructura de un programa se basa en bloques, que se definen mediante indentación: tabulación o 4 espacios.

```
def recorrido(listaEnteros):  
    listaPares = []  
    suma = 0  
    for elem in listaEnteros:  
        if elem % 2 == 0:  
            listaPares.append(elem)  
            suma += elem  
  
    return suma, listaPares  
  
# invocación  
sumatoria, listaPares = recorrido([1, 2, 3, 4])
```

```
def recorrido(listaEnteros):  
    return (sum(listaEnteros),  
            [x for x in listaEnteros if x % 2 == 0])  
  
# invocación  
sumatoria, listaPares = recorrido([1, 2, 3, 4])
```

Cuestiones generales del lenguaje

Las **variables** se declaran cuando aparecen por primera vez en el lado izquierdo en una asignación. Se utiliza tipado dinámico. No hay un constructor para definir **constantes**.

Las **funciones** se definen utilizando la palabra reservada **def**, seguido del nombre de la función y sus parámetros. Algunas facilidades que se brindan al usuario:

- Definir parámetros con valores por defecto.
- Pasar parámetros utilizando el nombre del parámetro formal.
- Una función puede asignarse a una variable, guardarse en una colección, o pasarse por parámetro.
- Una función siempre devuelve un valor (explícito utilizando *return*, o *None*).

```
def comparar(valor1, valor2):  
    if valor1 >= valor2:  
        return valor1  
    else:  
        print(valor2, "es mayor")  
  
>> x = 3  
⇒ None  
  
>> comparar(x, 2)  
⇒ 3  
  
>> comparar(3, 10.3)  
⇒ None  
  
>> comparar("lenguajes", "programación")  
⇒ None
```

Cuestiones generales del lenguaje

Una función puede tener parámetros formales con valores por defecto, lo que permite invocar la función con menos argumentos de los definidos.

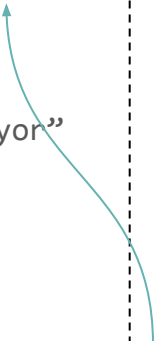
En una llamada, los parámetros pueden pasarse de forma posicional o utilizando el nombre del parámetro formal.

```
def comparar(valor1, valor2 = 0):  
    if valor1 >= valor2:  
        return valor1  
    else:  
        return "Valor2 es mayor"
```

```
>> comparar(3, 4)  
⇒ Valor2 es mayor
```

```
>> comparar(3)  
⇒ 3
```

Valor por defecto

A blue curved arrow originates from the text 'Valor por defecto' in a grey box at the bottom and points to the 'valor2 = 0' part of the function definition.

```
Model.fit(  
    x=None,  
    y=None,  
    batch_size=None,  
    epochs=1,  
    verbose="auto",  
    callbacks=None,  
    validation_split=0.0,  
    validation_data=None,  
    shuffle=True,  
    class_weight=None,  
    sample_weight=None,  
    initial_epoch=0,  
    steps_per_epoch=None,  
    validation_steps=None,  
    validation_batch_size=None,  
    validation_freq=1,  
)
```

Cuestiones generales del lenguaje

La sentencia *if-elif-else* nos permite evaluar distintas expresiones y tomar una decisión.

```
if (expresion_booleana) then
...
else
    if (expresion_booleana) then
        ...
    else
        if (expresion_booleana)
            then
                ...
            else ...
```



```
if expresion_booleana:
    ...
elif expresion_booleana:
    ...
elif expresion_booleana:
    ...
else:
    ...
```

Se puede consultar el valor de verdad de cualquier objeto para usarse en una instrucción condicional, iterativa o una expresión booleana. Por defecto, cualquier objeto es considerado True, salvo que se defina como False explícitamente, sea cero (valores numéricos) o tenga longitud cero (secuencias).

Cuestiones generales del lenguaje

La sentencia *while* permite iterar mientras la expresión booleana se evalúe a True.

```
while (expresion_booleana):  
    ... bloque del while
```

```
while (expresion_booleana_1) and (expresion_booleana_2):  
    ... bloque del while
```

```
while (expresion_booleana_1) or (expresion_booleana_2):  
    ... bloque del while
```

```
while not (expresion_booleana):  
    ... bloque del while
```

Cuestiones generales del lenguaje

Entre los principales operadores booleanos se encuentran el *or*, *and* y *not*, junto con los operadores de comparación.

Operador	Resultado	Ejemplos
x or y	Devuelve x si x es True, sino y.	$a > 0 \text{ or } b > 0 \Rightarrow \text{True/False}$ $[1, 2, 3] \text{ or } (b > 0) \Rightarrow [1, 2, 3]$ $\text{False or } 1 \Rightarrow 1$
x and y	Devuelve x si x es False, sino y.	$a > 0 \text{ and } b > 0 \Rightarrow \text{True/False}$ $[] \text{ and True} \Rightarrow []$ $1 \text{ and False} \Rightarrow \text{False}$
not x	Devuelve True si x es False, sino False.	not continuar

Entre los operadores de comparación se encuentran $<$, $>$, $<=$, $>=$, $==$, $!=$, *is* y *is not*.

Cuestiones generales del lenguaje

La sentencia *for* permite recorrer fácilmente cualquier iterador.

```
# recorrido en una lista de enteros
for i in range(10):
    print(i)
```

```
# recorrido en una lista
for item in [1, "hola", 3, 'a', 5]:
    print(item)
```

```
for item in miLista:
    print(item)
```

```
# recorrido en un string
for letra in "Lenguajes":
    print(letra)
```

```
# recorrido de las claves de un diccionario
for key in myDictionary.keys():
    print(key)
```

```
# recorrido de los valores de un diccionario
for value in myDictionary.values():
    print(value)
```

```
# recorrido de los items de un diccionario
for key, value in myDictionary.items():
    print(key, " -> ", value)
```

```
# recorrido en una lista agregando numeración
for indice, item in enumerate(miLista):
    print(indice, " -> ", item)
```

Python - Sistema de Tipos

Tipos de datos en Python

Python considera tipos básicos numéricos (enteros, reales y números complejos) y booleanos.

Además provee varios tipos estructurados, entre los que se encuentran:

- Listas
- Tuplas
- Conjuntos
- Diccionarios
- Strings

También se incluye la posibilidad de crear objetos a partir de clases definidas por el usuario.

```
entero = 10
real = 3.14
complejo = 2 + 3j
booleano = True
```

```
lista_numeros = [1, 2, 3, 4, 5]

tupla_colores = ("rojo", "verde", "azul")

mensaje = "Hola, mundo!"

diccionario_edades = {"Juan": 30,
                      "María": 25,
                      "Pedro": 35}

conjunto_vocales = {'a', 'e', 'i', 'o', 'u'}
conjunto_pares = set([2,4,6,8,10])
conjunto_impares = frozenset([1,3,5,7,9])
```

Clases definidas por el usuario

Al ser un lenguaje de scripting, Python admite la programación orientada a objetos. Permite implementar clases a partir de las cuales crear objetos. Cada clase tendrá sus atributos y métodos.

```
class Persona():  
    def __init__(self, nombre, edad):  
        self.nombre = nombre  
        self.edad = edad  
  
    def mostrar_nombre(self):  
        print(self.nombre)
```

La función `__init__` se invoca cada vez que se crea un objeto de la clase. Se utiliza para inicializar atributos y ejecutar métodos necesarios.

El parámetro `self` se utiliza para acceder a la instancia actual de la clase.

```
class Estudiante(Persona):  
    def __init__(self, nombre, edad):  
        Persona.__init__(nombre, edad)  
        self.institucion = 'UNS'  
  
    def mostrar_institucion(self):  
        print(self.institucion)
```

Una clase puede heredar de una clase padre.

Sistema de tipos :: Conversiones

Python considera algunas reglas de conversión implícita, y además provee conversiones explícitas mediante funciones definidas en el lenguaje.

Coerciones

- Para tipos numéricos, realiza conversiones hacia tipos más generales.
 - Un entero se convierte a real o un real a complejo.
- True y False se comportan como 1 y 0 si es necesario.

Conversiones explícitas

- Para tipos numéricos incluye int(), float(), complex().
- Para secuencias incluye list(), tuple(), set(), dict(), str().

Pese a estas funciones, no todas las conversiones serán permitidas.

```
x = 1
y = 1.1

>> x + y           ⇒ 2.1
>> x + True        ⇒ 2
>> x + False       ⇒ 1

>> int(y)          ⇒ 1
>> float(x)         ⇒ 1.0
>> complex(x)       ⇒ (1+0j)
>> str(y)           ⇒ '1.1'
>> list("hola")     ⇒ ['h','o','l','a']
>> dict([('a',1)])  ⇒ {'a': 1}
```

Sistema de tipos :: Conversiones

En general, cualquier objeto es considerado True, excepto en los siguientes casos:

- False
- None
- Secuencias y contenedores vacíos (listas, tuplas, cadenas, conjuntos, diccionarios).
- Ceros numéricos (enteros, reales, complejos).
- Tipos definidos por el usuario, explícitamente definidos como False usando el método `__bool__()`.

```
>> bool('')           ⇒ False
>> bool("hola")       ⇒ True
>> bool(set())         ⇒ False
>> bool(set([1, 2, 3])) ⇒ True
>> bool(None)          ⇒ False
```

```
class UserClass:
    def __init__(self, value):
        self.value = value

    def __bool__(self):
        return self.value > 0
```

```
obj1 = UserClass(0)
>> bool(obj1)    ⇒ False
```

```
obj2 = UserClass(1)
>> bool(obj2)    ⇒ True
```


Sistema de tipos :: Dureza en Expresiones Mixtas

Pese a las conversiones consideradas, no cualquier combinación de tipos son permitidas en el contexto de las expresiones.

Python	JavaScript
<pre>>> 1 + 2 ⇒ 3 >> 3.14 + 1 ⇒ 4.14</pre>	<pre>>> 1 + 2 ⇒ 3 >> 3.14 + 1 ⇒ 4.14</pre>
<pre>>> "hola" + "mundo" ⇒ "holamundo"</pre>	<pre>>> "hola" + "mundo" ⇒ "holamundo"</pre>
<pre>>> True + True ⇒ 2</pre>	<pre>>> true + true ⇒ 2</pre>
<pre>>> [1, 2, 3] + [4, 5, 6] ⇒ [1, 2, 3, 4, 5, 6] >> [1, 2, 3] + "5" ⇒ Error</pre>	<pre>>> [1, 2, 3] + [4, 5, 6] ⇒ "1,2,34,5,6" >> [1, 2, 3] + "5" ⇒ "1, 2, 35"</pre>
<pre>>> "Hola" + False ⇒ Error >> "5" + 1 ⇒ Error</pre>	<pre>>> "Hola" + false ⇒ "Holafalse" >> "5" + 1 ⇒ "51"</pre>
<pre>>> None + 5 ⇒ Error >> None + "hola" ⇒ Error >> None + [1, 2] ⇒ Error</pre>	<pre>>> NaN + 5 ⇒ NaN >> NaN + "hola" ⇒ "NaNhola" >> NaN + [1, 2] ⇒ "NaN1,2"</pre>

Sistema de tipos :: Sobrecarga de operadores

Python no permite sobrecarga de funciones, pero sí cuenta con operadores sobrecargados.

```
def f1(x):  
    print(x)
```

f1(4) ⇒ Imprime 4

```
def f1(x, y):  
    print(x)
```

f1(4) ⇒ Error (no existe función f1 de un parámetro)

```
suma = 10 + 20  
saludo = "Hola" + " " + "mundo."  
lista = [1, 2, 3] + [4, 5, 6]
```

```
producto = 5 * 3  
cadena = "Hola" * 3  
lista = [1, 2, 3] * 2
```

```
bool(10 <= 5)  
bool(10 != 10)  
bool("hola" < "mundo")  
bool("hola" != "mundo")  
bool([1, 2, 3] <= [4, 5, 6])
```

¿Por qué creen que no permite la sobrecarga de funciones?

Sistema de tipos :: Sobrecarga de operadores

Además, permite al usuario definir nuevas versiones de operadores predefinidos por el lenguaje.

```
class CustomList:
    def __init__(self, elements):
        self.elements = elements

    def __add__(self, other):
        if isinstance(other, int):
            other_list = [other]
        elif isinstance(other, list):
            other_list = other
        else:
            raise TypeError("Tipo no soportado")

        self.elements = self.elements + other_list
        return self

    def __repr__(self):
        return repr(self.elements)
```

```
# Ejemplo de uso
lista = CustomList([1, 2])
resultado = lista + 3
print(resultado)           ⇒ [1, 2, 3]
print(type(resultado)) ⇒ CustomList
```

Con buenas definiciones, se puede mejorar la facilidad de lectura y escritura.

Sistema de tipos :: Nivel de polimorfismo universal

Python brinda herencia simple y múltiple. Por otro lado, no soporta polimorfismo paramétrico.

```
class BaseClass:
    def m1(self):
        print("M1 de BaseClass")

class SubClass(BaseClass):
    def m1(self):
        print("M1 de SubClass")

base_obj = BaseClass()
sub_obj = SubClass()

base_obj.m1() ⇒ "M1 de BaseClass"
sub_obj.m1()  ⇒ "M1 de SubClass"
```

```
class BaseClass1:
    def m1(self):
        print("M1 de BaseClass1")

class BaseClass2:
    def m2(self):
        print("M2 de BaseClass2")

class SubClass(BaseClass1, BaseClass2):
    def m1(self):
        print("M1 de SubClass")
```

Sistema de tipos :: Nivel de polimorfismo universal

Una clase derivada no puede esconder atributos o métodos de una clase ancestro.

Una clase derivada puede definir comportamiento que no es compatible con el definido en su clase ancestro.

```
class CBase:
    def __init__(self):
        self.v1 = 1
        self.v2 = 2

    def get_message(self):
        return (self.v1, self.v2)

class CDerivada(CBase):
    def get_message(self):
        return 123
```

```
base_obj = CBase()
derived_obj = CDerivada()

len(base_obj.get_message())    ⇒ 2
len(derived_obj.get_message()) ⇒ Error
```

Si la relación es de tipo **is-a**, un objeto instancia de la subclase puede aparecer en cualquier lugar donde se espera un objeto de la clase padre, **sin causar un error de tipos**.

Sistema de tipos :: Control de tipos - Duck Typing

Python utiliza el concepto de ***duck typing*** para realizar parte de los chequeos de tipos. Al usar duck typing, los atributos y métodos de un objeto resultan más relevantes que el tipo del objeto.

```
class Pato:
    def hablar(self):
        print("Quack!")

class Persona:
    def hablar(self):
        print("Hola")

class Computadora:
    def encender(self):
        self.encendida = True
```

```
def hacer_hablar(x):
    x.hablar()
```

```
hacer_hablar(Pato())           ⇒ "Quack!"
hacer_hablar(Persona())       ⇒ "Hola"
hacer_hablar(Computadora())   ⇒ Error
```

Sistema de tipos :: Control de tipos - Duck Typing

Python utiliza el concepto de ***duck typing*** para realizar parte de los chequeos de tipos. Al usar duck typing, los atributos y métodos de un objeto resultan más relevantes que el tipo del objeto.

```
class Pato:
    def hablar(self):
        print("Quack!")

class Persona:
    def hablar(self):
        print("Hola")

class Computadora:
    def encender(self):
        self.encendida = True
```

```
def hacer_hablar(x):
    if hasattr(x, 'hablar') and callable(x.hablar):
        x.hablar()
    else:
        print("El objeto no habla")

hacer_hablar(Pato())           ⇒ "Quack!"
hacer_hablar(Persona())       ⇒ "Hola"
hacer_hablar(Computadora())   ⇒ "El objeto no habla"
```

Sistema de tipos :: Manejo de errores de tipos

Ante un error de tipos, Python lanzará una excepción correspondiente.

El lenguaje brinda los constructores necesarios para capturar las excepciones y que el programa pueda reponerse o terminar de manera controlada.

El usuario también puede lanzar una excepción en caso de que lo necesite.

```
def procesar_positivo(num):  
    if num < 0:  
        raise ValueError("Input debe ser >= 0")  
    print("Procesando...")  
  
try:  
    procesar_positivo(-1)  
except Exception:  
    print("Ocurrió un error...")
```


Caso de estudio: TypeScript

Introducción



TypeScript es un lenguaje de programación que extiende JavaScript, agregando tipado estático.

Es un superconjunto de JavaScript: cualquier programa válido en JavaScript también lo es en TypeScript.

Admite programación orientada a objetos y funcional.

El tipado es estático, lo cual permite detectar errores en tiempo de compilación.

```
function saludar(nombre: string): string {  
    return `Hola, ${nombre}`;  
}  
  
class Persona {  
    nombre: string;  
  
    constructor(nombre: string) {  
        this.nombre = nombre;  
    }  
  
    saludar(): void {  
        console.log(`Hola, soy ${this.nombre}.`);  
    }  
}  
  
const persona: Persona = new Persona("Ana");  
persona.saludar(); // Hola, soy Ana.
```

Introducción

TypeScript agrega tipado estático sobre JavaScript, lo que permite hacer chequeos de tipos antes de la ejecución. El código TypeScript se traduce a código JavaScript.

```
function saludar(nombre: string): string {  
    return `Hola, ${nombre}`;  
}  
  
class Persona {  
    nombre: string;  
  
    constructor(nombre: string) {  
        this.nombre = nombre;  
    }  
    saludar(): void {  
        console.log(`Hola, soy ${this.nombre}.`);  
    }  
}  
  
const persona: Persona = new Persona("Ana");  
persona.saludar(); // Hola, soy Ana.
```

TypeScript



```
"use strict";  
  
function saludar(nombre) {  
    return `Hola, ${nombre}`;  
}  
  
class Persona {  
    constructor(nombre) {  
        this.nombre = nombre;  
    }  
    saludar() {  
        console.log(`Hola, soy ${this.nombre}.`);  
    }  
}  
  
const persona = new Persona("Ana");  
persona.saludar(); // Hola, soy Ana.
```

JavaScript

Introducción

TypeScript busca mantener la semántica de JavaScript, pero hace controles para reconocer errores de tipos que en JavaScript no se reconocen.

```
function sum(a, b) {  
  return a + b;  
}  
  
sum(1, 2); // 3  
sum(2, 2); // 4  
sum(0, 'string'); // '0string'
```

JavaScript

```
function sum(a: number, b: number): number {  
  return a + b;  
}  
  
sum(1, 2); // 3  
sum(2, 2); // 4  
sum(0, 'string'); // Argument of type '"string"' is not  
                   assignable to parameter of type 'number'.
```

TypeScript

Menos flexibilidad en favor de mayor fiabilidad.

Cuestiones generales del lenguaje

Las **variables** se declaran usando las palabras clave **let** o **var**. Para definir **constantes**, se utiliza **const**.

TypeScript permite especificar el tipo de las variables al momento de la declaración. Si no se especifica, **el tipo se puede inferir automáticamente** a partir de los valores.

El tipado es estático, por lo que los tipos se comprueban en tiempo de compilación.

```
let edad_1;  
let edad_2 = 30;  
let edad_3 : number = 30;  
  
edad_1 = 10;  
  
const nombre = "Lucía";  
  
function saludar(nombre: string) {  
    return `Hola, ${nombre}`;  
}  
  
const saludo = saludar(nombre); // "Hola, Lucía"
```

TypeScript

```
declare let edad_1: any;  
declare let edad_2: number;  
declare let edad_3: number;  
  
  
  
declare const nombre: string;  
  
declare function saludar(nombre: string): string;  
  
declare const saludo: string;
```

.D.TS

Cuestiones generales del lenguaje

Las **funciones** se definen utilizando la palabra **function**, con anotaciones de tipo para los parámetros y el valor de retorno.

En algunos casos, el lenguaje puede inferir los tipos involucrados.

- Se pueden definir parámetros con valores por defecto.
- Las funciones pueden asignarse a variables, almacenarse en colecciones o pasarse como argumento.

```
const nombre: string = "Lucía";

function multiplicar(a: number, b: number): number {
  return a * b;
}

function saludar(nombre: string = "Invitado"): string {
  return `Hola, ${nombre}`;
}

const saludo = saludar();
console.log(saludo); // "Hola, Invitado"

const saludo = saludar(nombre);
console.log(saludo); // "Hola, Lucía"
```

Cuestiones generales del lenguaje

TypeScript ofrece los mismos constructores de selección que JavaScript: *if-else if-else*, y *switch*.

```
let edad: number = 20;

if (edad >= 18) {
  console.log("Mayor de edad");
} else {
  console.log("Menor de edad");
}
```

```
let dia: string = "lunes";

switch (dia) {
  case "lunes":
    console.log("Inicio de semana");
    break;
  case "viernes":
    console.log("Fin de semana cerca");
    break;
  default:
    console.log("Día cualquiera");
}
```

Las expresiones booleanas pueden incluir comparaciones (==, ===, !=, !==, <, >, etc.) y operadores lógicos (&&, ||, !). El valor *false*, 0, "", *null*, *undefined* y *NaN* se consideran falsos en contextos booleanos.

Cuestiones generales del lenguaje

Para iteración, se incluyen constructores *while*, *for*, *do-while*, *for-of* y *for-in*.

```
for (let i: number = 1; i <= 5; i++)  
{  
  console.log(`Número: ${i}`);  
}
```

```
let i: number = 1;  
  
while (i <= 5) {  
  console.log(`Número: ${i}`);  
  i++;  
}
```

```
let i: number = 1;  
do {  
  console.log(`Número: ${i}`);  
  i++;  
} while (i <= 5);
```


Cuestiones generales del lenguaje

Para iteración, se incluyen constructores *while*, *for*, *do-while*, *for-of* y *for-in*.

- *for...of* permite recorrer los valores de una colección iterable (como un array, string, etc).

```
const frutas: string[] = ["manzana", "banana", "naranja"];

for (const fruta of frutas) {
  console.log(fruta);
}
```

- *for...in* permite recorrer las claves (índices o propiedades) de un objeto.

```
const persona = { nombre: "Ana", edad: 30 };

for (const clave in persona) {
  console.log(`${clave}: ${persona[clave]}`);
}
```

JavaScript

```
type Persona = {nombre: string, edad: number};
const persona: Persona = { nombre: "Ana", edad: 30 };

for (const clave in persona) {
  console.log(`${clave}: ${persona[clave as keyof Persona]}`);
}
```

TypeScript

TypeScript - Sistema de Tipos

Sistema de tipos :: Tipos de datos en TypeScript

TypeScript soporta los mismos tipos de datos que JavaScript, pero añade anotaciones de tipo para mayor seguridad y control.

Tipos básicos:

- number, string, boolean, bigint, symbol, null, undefined.

Tipos estructurados:

- arreglos, tuplas, enumerados, diccionario, unión, intersección.

También incluye algunos tipos especiales:

- void, any, unknown, never.

```
// Arreglo
let frutas: string[] = ["manzana", "pera", "banana"];

// Tupla
let persona: [string, number] = ["Ana", 30];

// Objeto
let usuario: { nombre: string; edad: number } = {
  nombre: "Luis",
  edad: 25,
};

// Enumerado
enum Direccion {
  Norte,
  Sur,
  Este,
  Oeste,
}

let rumbo: Direccion = Direccion.Norte;

// Unión de tipos: puede ser número o null
let puntuacion: number | null = null;
```

Sistema de tipos :: Tipos definidos por el usuario

Además, el lenguaje incluye un constructor *type* y un constructor *interface*.

```
function printCoord(pt:{ x:number, y:number }) {  
  console.log(pt.x, pt.y);  
}  
  
let point = { x: 100, y: 100 };  
printCoord(point);
```



```
type Point = {  
  x: number; y: number;  
};  
  
function printCoord(pt: Point) {  
  console.log(pt.x, pt.y);  
}  
  
let point = { x: 100, y: 100 };  
printCoord(point);
```

```
interface Point {  
  x: number; y: number;  
};  
  
function printCoord(pt: Point) {  
  console.log(pt.x, pt.y);  
}  
  
let point = { x: 100, y: 100 };  
printCoord(point);
```

Sistema de tipos :: Clases definidas por el usuario

TypeScript admite la programación orientada a objetos, permite implementar clases a partir de las cuales crear objetos. Cada clase tendrá sus atributos y métodos.

```
class Persona {  
  nombre: string;  
  edad: number;  
  
  constructor(nombre: string, edad: number) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  
  saludar(): void {  
    console.log(`Hola, soy ${this.nombre} y tengo ${this.edad} años.`);  
  }  
}
```

```
const persona1 = new Persona("Menguechito", 30);  
persona1.saludar(); // Hola, soy Menguechito y tengo 30 años.
```

Sistema de tipos :: Conversiones

En JavaScript se cuenta con múltiples coerciones, donde el operador tendrá una gran influencia sobre las conversiones de los datos. El objetivo es evitar que el programa se detenga por un error de tipos, para lo cual los operandos mutan hacia el tipo de dato que manda el contexto.

Tipos que se encuentran	Operador	Resultado	Ejemplo
boolean y número	cualquiera	boolean → número	True + 14 ⇒ Res: 15
string y número	+	número → string (concatena)	"Hola" + 1 ⇒ Res: "Hola1"
string y número	-, *, /, %	string → número (opera)	"42" - 1 ⇒ Res: 41
objeto y primitivo	cualquiera	objeto → primitivo (usando valueOf/toString)	var caja = {manzanas : 4, valueOf(){...}} caja + 1 ⇒ Res: 5
null / undefined	==	solo iguales entre sí	null == undefined ⇒ Res: True

Sistema de tipos :: Conversiones

TypeScript hereda las coerciones de JavaScript, por lo tanto permite conversiones implícitas en tiempo de ejecución.

Sin embargo, en tiempo de compilación puede advertir sobre usos que típicamente se consideran errores y en los cuales las coerciones de JavaScript no deberían llevarse a cabo.

```
let v: any = "123";  
v = v + 1;  
console.log(v); // "1231" (string)  
  
v = v * 2;  
console.log(v); // 246 (number)
```

Se ejecuta sin errores ni advertencias

```
let v: string = "123";  
v = v + 1;  
console.log(v); // "1231" (string)  
  
v = v * 2;  
console.log(v); // 246 (number)
```

Se ejecuta pero TypeScript marca que hay un error de tipos

Sistema de tipos :: Sobrecarga de funciones

TypeScript permite sobrecarga de funciones. Para ello, se definen varias firmas, y una implementación única, compatible con todas las firmas, capaz de manejar cualquier invocación.

```
function combinar(a: string, b: string): string;
function combinar(a: number, b: number, c: number): number;
function combinar(a: any, b: any, c?:number): any {
    if (c !== undefined){
        return a+b+c;
    }
    return a+b;
}

const s = combinar("Hola, ", "mundo"); // string
const n = combinar(5, 10, 15);          // number
```

En JavaScript no existe sobrecarga de funciones.

Sistema de tipos :: Nivel de polimorfismo universal

TypeScript cuenta con herencia simple (no múltiple).

Al contar con tipos, incluye polimorfismo paramétrico que puede usarse en funciones o clases.

```
class Contenedor<T> {  
  valor: T;  
  constructor(v: T) {  
    this.valor = v;  
  }  
  
  obtener(): T {  
    return this.valor;  
  }  
}
```

```
class ContenedorConNombre<T> extends Contenedor<T> {  
  nombre: string;  
  
  constructor(v: T, nombre: string) {  
    super(v);  
    this.nombre = nombre;  
  }  
  
  describir(): string {  
    return `${this.nombre} contiene ${this.valor}`;  
  }  
}
```

```
// Uso  
const c = new ContenedorConNombre<number>(42, "Caja");  
console.log(c.describir()); // "Caja contiene 42"
```

Sistema de tipos :: Control de tipos

Una vez que se conocen los tipos de todos los elementos (declarados, inferidos o siendo de tipo *any*), el lenguaje verifica que el uso sea el correcto.

- En comparación con JavaScript, se restringen las coerciones, lo que define qué expresiones son válidas y qué expresiones no lo son.
- Se verifican asignaciones, llamadas a funciones, expresiones, etc.
- Los tipos (no primitivos) son compatibles por estructura, no por nombre.

```
type Point = { x: number; y: number };

type ValuePair = { x: number; y: number };

function mostrar(p: Point) {
  console.log(p.x, p.y);
}

const obj = { x: 10, y: 20};
mostrar(obj); // 10 20

const obj2: ValuePair = { x: 11, y: 21};
mostrar(obj2); // 11 21

const obj3 = { x: 12, y: 22, z:32};
mostrar(obj3); // 12 22

const obj4 = { x: 13 };
mostrar(obj4); // Error en compilación
```

Sistema de tipos :: Manejo de errores de tipos

TypeScript tiene un mecanismo de manejo de excepciones similar al de JavaScript, usando las palabras clave *try*, *catch*, *finally*. La sintaxis y el funcionamiento son los mismos que en JavaScript.

```
function procesarPositivo(num: number): void {  
  if (num < 0) {  
    throw new Error("Input debe ser >= 0");  
  }  
  console.log("Procesando...");  
}  
  
try {  
  procesarPositivo(-1);  
} catch (error) {  
  console.log("Ocurrió un error...");  
}
```

¿Preguntas?
